7.Distributed Database Design

To serve a global audience, the system utilizes a distributed database architecture that spreads data across multiple geographic locations. This "partitioning" or "sharding" of data ensures that a user in London experiences the same speed as a user in New York by accessing a local data node. We use "geographic replication," where the database is cloned across different continents to provide high availability; if one data center goes offline, another takes over instantly. Managing the "CAP theorem" trade-off is central to this design—balancing Consistency, Availability, and Partition Tolerance. For flight availability, we prioritize "Strong Consistency" to avoid overbooking, while for user reviews or flight history, we might allow "Eventual Consistency" to boost performance. We also implement "Global Load Balancing" to direct traffic to the healthiest and closest database server available. This distributed approach also makes the system more "elastic," meaning we can add more servers during high-demand periods like the summer holidays. It reduces the "single point of failure" risk, ensuring that a localized power outage doesn't bring down the global booking network. This sophisticated setup is essential for modern airlines that operate across different time zones and require 24/7 operational capability.

8.Failure Recovery

The failure recovery module is the system's "insurance policy" against hardware crashes, power failures, or human errors. We implement "Write-Ahead Logging" (WAL), which records every change to a log file before it is permanently written to the main database. If the system crashes, it can replay these logs to restore the database to its exact state just before the failure. We also use a "Point-in-Time Recovery" (PITR) strategy, allowing us to "roll back the clock" to a specific minute if a major data corruption occurs.

: Regular "Full Backups" are taken daily, while "Incremental Backups" capture changes every hour to minimize data loss. These backups are stored in multiple, physically separate locations to protect against natural disasters at the primary data center. "Health Checks" and "Auto-Failover" mechanisms are in place to detect a server failure and switch to a "Standby" database in seconds. We also practice "Recovery Drills" to ensure that our team can restore the system within the promised "Recovery Time Objective" (RTO). "Check-pointing" is used to clear out old logs and keep the recovery process fast and manageable. By preparing for the worst-case scenario, we ensure that the airline's data—and the passengers' travel plans—are always protected.

9.Methodology

The development process follows a rigorous "Software Development Life Cycle" (SDLC), starting with an intensive requirement gathering phase. We use an "Agile" approach, breaking the project into two-week "sprints" where specific features like the search engine or payment portal are built and tested. For the database layer, we use MySQL due to its proven track record in handling high-concurrency web applications. "ER Workbench" is our primary tool for visual modeling, allowing us to spot design flaws before a single line of code is written. We implement "Unit Testing" for every database function and "Integration Testing" to ensure the frontend and backend communicate perfectly. "Stress Testing" is conducted by simulating 10,000 simultaneous users to see where the system might break under extreme load. We also use "Version Control" (like Git) for our database scripts, ensuring that every change is tracked and can be reversed if necessary. The methodology includes a "User Acceptance Testing" (UAT) phase where actual airline staff provide feedback on the system's usability. This structured approach reduces bugs and ensures that the final product perfectly aligns with the initial objectives. By combining modern tools with disciplined engineering practices, we guarantee a high-quality delivery.